Experiment No: 08

Course Code: CSE-406
Course Title: Digital Image Processing Labratory
4th Year 1st Semester Examination 2023

Date of Performance: 20-10-2024
Date of Submission: 27-10-2024

Submitted to-

Dr. Morium Akter
Professor
Dr. Md. Golam Moazzem
Professor

Department of Computer Science and Engineering
Jahangirnagar University
Savar, Dhaka-1342

Sl	Class Roll	Exam Roll	Name
01	364	202176	Suraiya Mahmuda

Experiment Name:

a. **How will an image look after adding Gaussian, Raylegh and Erlang Noise.**
b. **Image corruption by Gassian Noise after 3*3 Arithmetic and Geometric Mean Filter.**
c. **Removal of Salt and Pepper Noise.**
d. **Periodic Noise Removal.**

a. Objectives:

To demonstrate how different types of noise affect an image.

- **Gaussian Noise**: This noise appears as random variations in brightness across the image, causing a grainy effect.
- **Rayleigh Noise**: Typically appears in images with a lot of low-intensity details, leading to random bright spots and variations.
- **Erlang Noise**: Similar to Gaussian but has a heavier tail, often making it more prominent in certain regions.

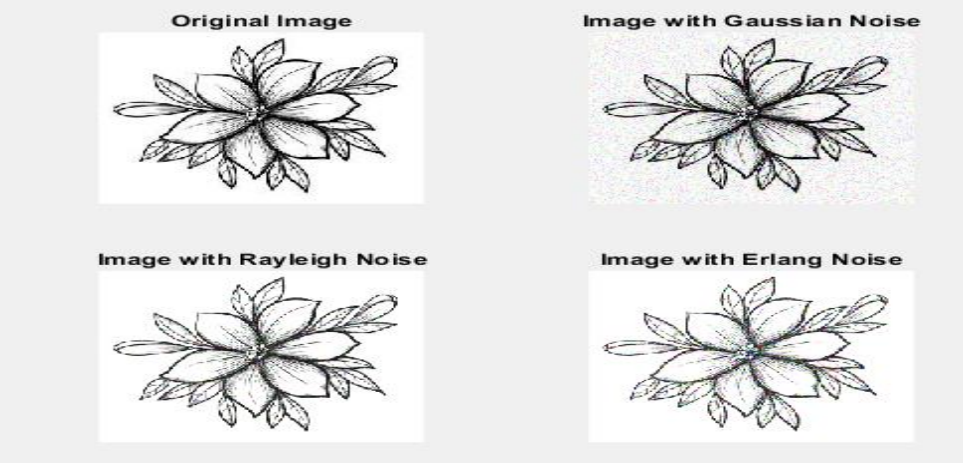
After adding these noises, the image will appear distorted, with variations in pixel intensity that degrade the overall quality**.**

Source Code:

```
% Read an image
img = imread('C:\Users\LAB203\Desktop\Input.jpg'); % Replace with your image file
img = im2double(img); % Convert to double for processing
% Add Gaussian Noise
mean_g = 0; % Mean of Gaussian noise
var_g = 0.01; % Variance of Gaussian noise
gaussian_noise = mean_g + sqrt(var_g) * randn(size(img));
img_gaussian = img + gaussian_noise;
% Add Rayleigh Noise
scale_r = 0.1; % Scale parameter for Rayleigh noise
rayleigh_noise = raylrnd(scale_r, size(img));
img_rayleigh = img + rayleigh_noise;
% Add Erlang Noise
k = 2; % Shape parameter (k > 0)
scale_e = 0.1; % Scale parameter for Erlang noise
erlang_noise = gamrnd(k, scale_e, size(img)); % Gamma distribution for Erlang
img_erlang = img + erlang_noise;
% Ensure that the images remain in the range [0, 1] after adding noise
img_gaussian = min(max(img_gaussian, 0), 1);
img_rayleigh = min(max(img_rayleigh, 0), 1);
img_erlang = min(max(img_erlang, 0), 1);
% Display images in one figure
figure;
% Original Image
subplot(2, 2, 1); % 2 rows, 2 columns, 1st subplot
imshow(img);
title('Original Image');
% Image with Gaussian Noise
subplot(2, 2, 2); % 2 rows, 2 columns, 2nd subplot
imshow(img_gaussian);
title('Image with Gaussian Noise');
```

```
% Image with Rayleigh Noise
subplot(2, 2, 3); % 2 rows, 2 columns, 3rd subplot
imshow(img_rayleigh);
title('Image with Rayleigh Noise');
% Image with Erlang Noise
subplot(2, 2, 4); % 2 rows, 2 columns, 4th subplot
imshow(img_erlang);
title('Image with Erlang Noise');
```

Input and Output:



b. Objectives:

To show the effect of filtering techniques after adding noise.

- **Gaussian Noise Addition**: The image becomes grainy.
- **Arithmetic Mean Filter**: This filter averages pixel values in a 3×33 \times 33×3 neighborhood. The result is a smoothed image but may lose finer details.
- **Geometric Mean Filter**: This filter preserves edges better than the arithmetic mean filter, resulting in a smoother image while retaining more detail.

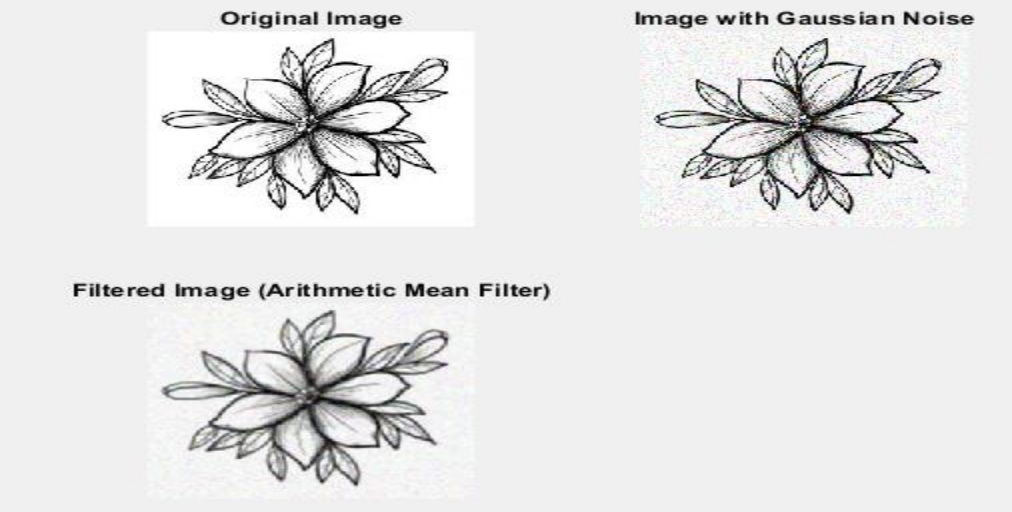
After applying both filters, the image will appear less noisy, with the arithmetic mean resulting in a more blurred effect compared to the more detailed preservation from the geometric mean.

Source Code:

```
% Read an image
img = imread('C:\Users\CSELAB-2\Desktop\Labb8\Input.jpg'); % Replace with your image file
img = im2double(img); % Convert to double for processing
% Display original image
figure;
% Original Image
subplot(2, 2, 1); % 2 rows, 2 columns, 1st subplot
imshow(img);
title('Original Image');
% Add Gaussian Noise
mean_g = 0; % Mean of Gaussian noise
```

```
var_g = 0.01; % Variance of Gaussian noise
gaussian_noise = mean_g + sqrt(var_g) * randn(size(img)); % Generate Gaussian noise
noisy_img = img + gaussian_noise; % Corrupt the image with noise
% Ensure the noisy image remains within the valid range [0, 1]
noisy_img = min(max(noisy_img, 0), 1);
% Display image with Gaussian Noise
subplot(2, 2, 2); % 2nd subplot
imshow(noisy_img);
title('Image with Gaussian Noise');
% Apply 3x3 Arithmetic Mean Filter
mean_filter = fspecial('average', 3); % Create a 3x3 mean filter
filtered_mean_img = imfilter(noisy_img, mean_filter, 'replicate'); % Apply filter
% Display filtered image using Arithmetic Mean Filter
subplot(2, 2, 3); % 3rd subplot
imshow(filtered_mean_img);
title('Filtered Image (Arithmetic Mean Filter)');
% Apply 3x3 Geometric Mean Filter
% Initialize the filtered image
filtered_geom_img = zeros(size(noisy_img));
% Apply Geometric Mean Filter
for i = 1:size(noisy_img, 1)
   for j = 1:size(noisy_img, 2)
% Get the neighborhood
neighborhood = noisy_img(max(i-1, 1):min(i+1, size(noisy_img, 1)), ...
max(j-1, 1):min(j+1, size(noisy_img, 2)), :);
% Calculate the geometric mean for each channel
filtered_geom_img(i, j, :) = exp(mean(log(neighborhood + eps), [1 2])); % +eps to avoid log(0)
   end
end
% Display filtered image using Geometric Mean Filter
subplot(2, 2, 4); % 4th subplot
imshow(filtered_geom_img);
title('Filtered Image (Geometric Mean Filter)');
```

Input and Output:



c. Objectives:

To demonstrate noise reduction techniques for salt and pepper noise.

- **Salt and Pepper Noise**: This noise manifests as randomly occurring white and black pixels.
- **Removal Technique**: Typically achieved using a median filter, which effectively removes this type of noise by replacing each pixel with the median value of its neighbors.

After applying the median filter, the image should look cleaner, with significantly fewer noisy pixels.

Source Code:

```
% Read an image
img = imread('C:\Users\LAB203\Desktop\Input.jpg'); % Replace with your image file
img = im2double(img); % Convert to double for processing
% Add Salt and Pepper Noise
noise_density = 0.05; % Density of noise
noisy_img = imnoise(img, 'salt & pepper', noise_density);
% Remove Salt and Pepper Noise using Median Filter
filtered_img = img; % Initialize filtered image
if size(noisy_img, 3) == 3
   % If the image is RGB, apply median filter to each channel
   filtered_img = zeros(size(noisy_img)); % Initialize filtered image for RGB
   for i = 1:3
      filtered_img(:,:,i) = medfilt2(noisy_img(:,:,i)); % Apply median filter to each channel
   end
else
   % For grayscale image
   filtered_img = medfilt2(noisy_img);
end
% Display images in one figure
figure;
% Original Image
subplot(1, 3, 1); % 1 row, 3 columns, 1st subplot
imshow(img);
title('Original Image');
% Image with Salt and Pepper Noise
subplot(1, 3, 2); % 1 row, 3 columns, 2nd subplot
imshow(noisy_img);
title('Image with Salt and Pepper Noise');
% Filtered Image
subplot(1, 3, 3); % 1 row, 3 columns, 3rd subplot
imshow(filtered_img);
title('Filtered Image (Median Filter)');
```

Input and Output:

d. Objectives: To illustrate the process of removing periodic noise using frequency domain techniques.

- **Periodic Noise**: Appears as regular patterns or stripes in an image.
- **Removal Technique**: The process involves using the Fourier Transform to identify and filter out the periodic components.

After processing, the image should show reduced regular patterns, restoring the visual integrity of the original image.

Source Code:

```
% Read an image
img = imread('C:\Users\CSELAB-2\Desktop\Labb8\Input.jpg'); % Replace with your image file
img = im2double(img); % Convert to double for processing
% Convert to grayscale if the image is RGB
if size(img, 3) == 3
   img_gray = rgb2gray(img);
else
   img_gray = img;
end
% Perform FFT
F = fft2(img_gray);
F_shifted = fftshift(F); % Shift zero frequency components to center
% Create a filter to remove periodic noise
[m, n] = size(img_gray);
cutoff_radius = 30; % Adjust as necessary for your image
[x, y] = meshgrid(-n/2:n/2-1, -m/2:m/2-1);
distance = sqrt(x.^2 + y.^2);
% Create a circular high-pass filter
hp_filter = double(distance > cutoff_radius);
% Apply the filter in the frequency domain
F_filtered = F_shifted .* hp_filter;
% Inverse FFT to get the filtered image
filtered_F = ifftshift(F_filtered); % Shift back
img_filtered = ifft2(filtered_F);
img_filtered = real(img_filtered); % Take the real part
% Display input and output images in one figure
figure;
% Input Image
subplot(1, 2, 1); % 1 row, 2 columns, 1st subplot
imshow(img_gray);
title('Original Image');
% Filtered Image
subplot(1, 2, 2); % 1 row, 2 columns, 2nd subplot
imshow(img_filtered, []);
title('Filtered Image (Periodic Noise Removed)');
```

Input and Output: